PYTHON ACTIVITY-BASED LEARNING

REPORT ON 10 SELECTED CASE STUDIES

Submitted in partial fulfillment of the requirements for the subject Python Programming

Student Details:
Name: G. maneesh satya | Roll Number: 26000909
Branch: CSE (AI/ML) | Section: 1 | Subject: Python Programming
Academic Year: 2026–2027 | Project Title: Implementation of Ten Python Case Studies
GitHub Repository: Python-Activity-Based-Learning

INTRODUCTION

Activity-Based Learning empowers students to comprehend programming paradigms by applying fundamental concepts to realistic problem domains. This project compiles ten distinct Python case studies demonstrating rigorous computational problem-solving.

The selected case studies cover arithmetic operations, conditionals, iterative loops, string manipulation, lists, nested lists, tuples, and data processing. Each case study provides a structured breakdown encompassing a problem statement, objectives, assumptions, algorithm, monochrome flowchart, pseudocode, Python source code, test cases, and conclusion.

OBJECTIVES OF THE PROJECT

1. To understand computational thinking and structured problem-solving methodologies.
2. To design clear algorithms and pseudocode for logical problem statements.
3. To master Python variables, data types, operators, and conditional branching.
4. To implement iteration structures, string functions, lists, nested matrices, and tuples.
5. To execute Python scripts and rigorously verify program correctness using test cases.
6. To maintain well-structured source code repositories aligned with academic standards.

CASE STUDY 1: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Develop a Python program to simulate a sequence of ATM transactions (deposits and withdrawals) maintaining account balances, validating amounts, and summarizing accepted versus rejected transactions.

2. Objectives

- To process financial transactions sequentially using loops and collections.
- To apply conditional validation for deposits and withdrawals against balance constraints.

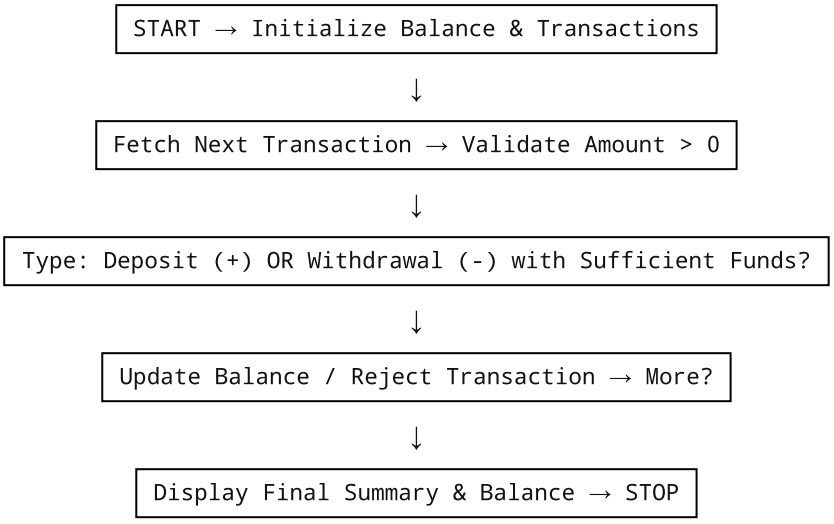
3. Assumptions

Opening balance is non-negative; transaction amounts are expressed in numeric currency; withdrawals exceeding balance are rejected.

4. Algorithm

1. Start program; initialize account balance and transaction list.
2. Iterate through each transaction tuple `(action, amount)`.
3. Validate amount > 0 and check sufficiency of funds for withdrawals.
4. Update balance for valid transactions; increment rejection counters otherwise.
5. Display transaction summary and final balance. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
balance = float(input("Enter opening balance: "))
transactions = [("deposit", 1500), ("withdraw", 400), ("withdraw", 5000), ("deposit", 750), ("wit
hdraw", 250)]
accepted = 0; rejected = 0
for action, amount in transactions:
    if amount <= 0:
        print("Invalid transaction amount"); rejected += 1
    elif action == "deposit":
        balance += amount; accepted += 1
        print("Deposited:", amount, "Balance:", balance)
    elif action == "withdraw":
        if amount <= balance:
            balance -= amount; accepted += 1
            print("Withdrawn:", amount, "Balance:", balance)
        else:
            print("Insufficient balance for:", amount); rejected += 1
print("Accepted:", accepted, "| Rejected:", rejected, "| Final Balance:", balance)
```

CASE STUDY 2: ADVANCED NUMERIC PATTERN GENERATOR

1. Problem Statement

Create a Python program to generate advanced structured numeric patterns (such as Floyd's Triangle or custom number pyramids) based on user-defined row counts using nested loops.

2. Objectives

- To master nested loop execution and formatted string printing in Python.
- To control spacing and sequence generation mathematically.

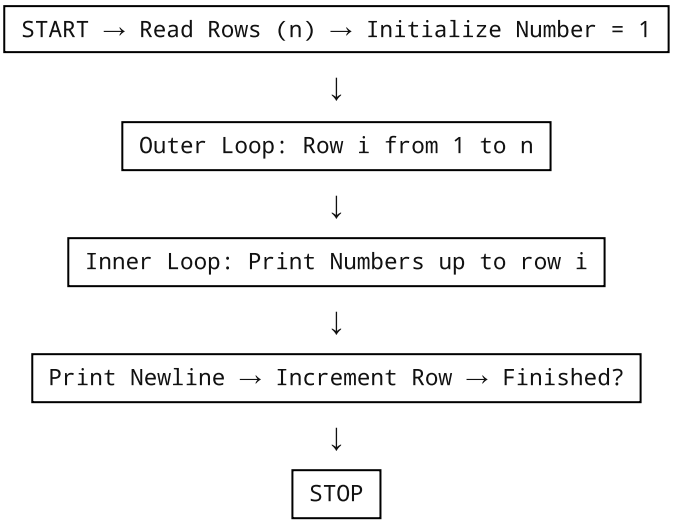
3. Assumptions

The input row count is a positive integer greater than zero.

4. Algorithm

1. Start program; read total rows `n`.
2. Initialize a counter variable for sequential numbering.
3. Use an outer loop for rows (1 to n) and an inner loop for column printing.
4. Print numbers sequentially with spacing. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
rows = int(input("Enter number of rows for pattern: "))
if rows <= 0:
    print("Invalid row count. Must be positive.")
else:
    num = 1
    print("Generated Numeric Pattern (Floyd's Triangle style):")
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(num, end=" ")
            num += 1
        print()
```

7. Test Cases & Conclusion

Test Cases: (1) Input rows = 4 (Outputs 10 numbers in 4 rows). (2) Input rows = 0 (Displays validation error message).

Conclusion: Demonstrates nested loop control flow and formatted sequential number generation.

CASE STUDY 3: REPEATED TRANSACTION DETECTION

1. Problem Statement

Develop a program to scan transaction logs, identify duplicate transaction IDs, and flag high-value transactions exceeding a specified financial threshold.

2. Objectives

- To use lists and membership checks (`in`, `not in`) for duplicate identification.
- To combine conditional threshold logic with iterative scanning.

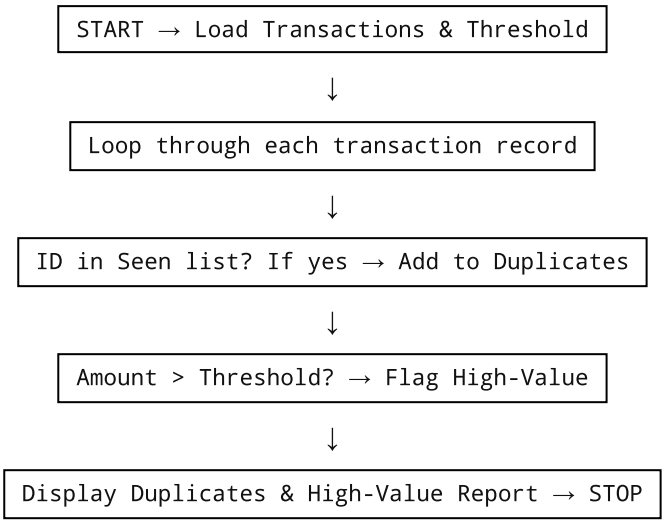
3. Assumptions

Transactions are provided as a list of tuples containing transaction ID and amount.

4. Algorithm

1. Start; define transactions list and threshold value.
2. Initialize `seen` and `duplicates` tracking lists.
3. Iterate through records; check if ID exists in `seen`. If yes and not in `duplicates`, add to `duplicates`.
4. Check if amount exceeds threshold. Display summary. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
transactions = [("TX101", 450), ("TX102", 1200), ("TX103", 850), ("TX102", 1200), ("TX104", 620
0), ("TX105", 300)]
seen = []; duplicates = []; threshold = 5000
for tid, amount in transactions:
    if tid in seen:
        if tid not in duplicates: duplicates.append(tid)
    else:
        seen.append(tid)
    if amount > threshold:
        print("High-value transaction flagged:", tid, "Amount:", amount)
print("Duplicate Transaction IDs Found:", duplicates if duplicates else "None")
```

7. Test Cases & Conclusion

Test Cases: (1) List with duplicate TX102 and high-value TX104. (2) Unique transaction set.

Conclusion: Demonstrates membership searching and duplicate detection in structured data lists.

CASE STUDY 4: PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Create a Python program to validate user passwords against strict security policies: minimum length of 8 characters, inclusion of uppercase, lowercase, digits, and special characters.

2. Objectives

- To apply string inspection methods (`isupper`, `islower`, `isdigit`).
- To enforce complex conditional logic for security compliance.

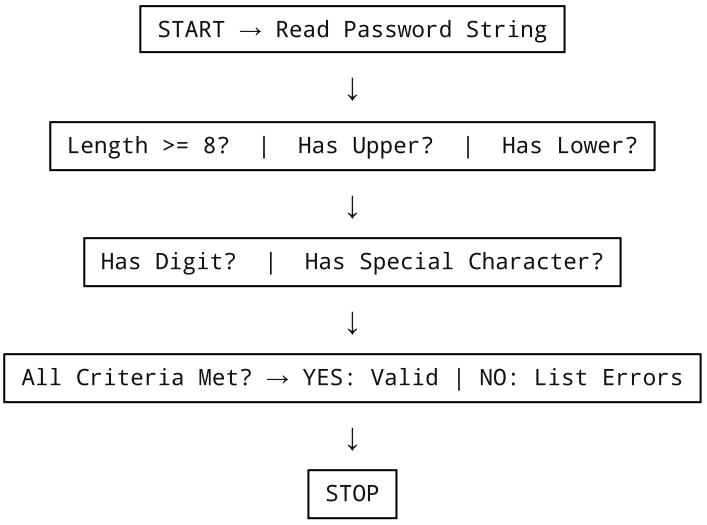
3. Assumptions

Password is entered as a standard string.

4. Algorithm

1. Start program; read password string.
2. Check length >= 8.
3. Scan characters for at least one uppercase, lowercase, digit, and special character.
4. Display validation success or list missing policy requirements. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
pwd = input("Enter password to validate: ")
errors = []
if len(pwd) < 8: errors.append("Length must be at least 8 characters")
if not any(c.isupper() for c in pwd): errors.append("Missing uppercase letter")
if not any(c.islower() for c in pwd): errors.append("Missing lowercase letter")
if not any(c.isdigit() for c in pwd): errors.append("Missing digit")
if not any(not c.isalnum() for c in pwd): errors.append("Missing special character")

if not errors:
    print("Password Policy Validation: PASSED (Strong Password)")
else:
    print("Password Policy Validation: FAILED")
    for err in errors: print("-", err)
```

7. Test Cases & Conclusion

Test Cases: (1) `Python@2026` (Passed). (2) `weakpwd` (Failed: missing upper, digit, special, length).

Conclusion: Demonstrates string inspection methods and multi-condition security validation.

CASE STUDY 5: STRING-BASED EXAMINATION ANSWER EVALUATION

1. Problem Statement

Develop a Python program that compares student answer strings with an expected answer key, accounting for correct answers, wrong answers, and unanswered questions with scoring penalties.

2. Objectives

- To utilize string comparison and sanitization (`strip`, `upper`).
- To implement scoring formulas with positive and negative marking.

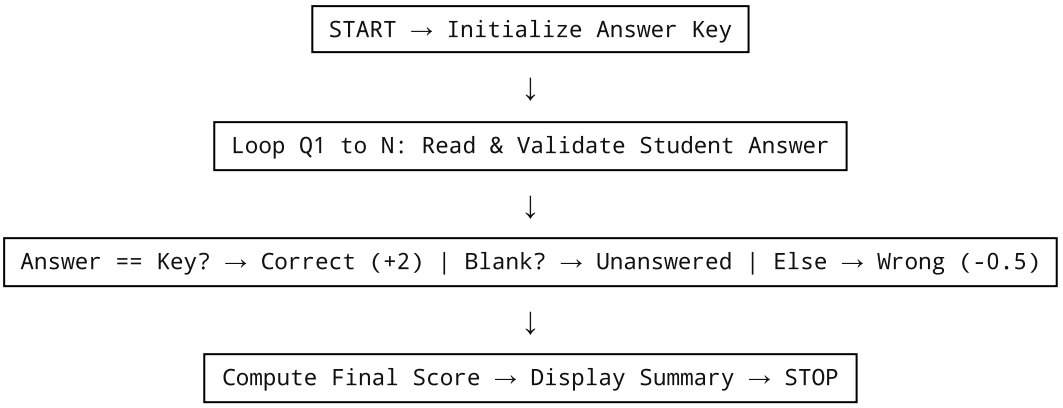
3. Assumptions

Answer keys and student submissions are matched index-by-index.

4. Algorithm

1. Start program; define answer key list.
2. Read student answers iteratively with input validation.
3. Compare each student answer with the key. Count correct, wrong, and blank.
4. Calculate final score using formula: `(correct * 2) - (wrong * 0.5)`. Display summary. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
answer_key = ["A", "C", "B", "D", "A"]
answers = ["A", "B", "B", "", "A"] # Sample student submission
correct = 0; wrong = 0; unanswered = 0
for i in range(len(answer_key)):
    ans = answers[i].strip().upper()
    if ans == "": unanswered += 1
    elif ans == answer_key[i]: correct += 1
    else: wrong += 1
score = (correct * 2) - (wrong * 0.5)
print("Exam Results -> Correct:", correct, "| Wrong:", wrong, "| Unanswered:", unanswered)
print("Final Score:", score, "out of", len(answer_key) * 2)
```

7. Test Cases & Conclusion

Test Cases: (1) All correct answers. (2) Mixed answers with blanks and wrong entries.

Conclusion: Demonstrates string comparison, sanitization, and automated grading calculation.

CASE STUDY 6: TEXT COMPRESSION USING CHARACTER RUNS

1. Problem Statement

Implement Run-Length Encoding (RLE) text compression in Python to compress repetitive character strings by replacing consecutive identical characters with character-count pairs.

2. Objectives

- To apply string traversal and counting algorithms.
- To manipulate string concatenation efficiently.

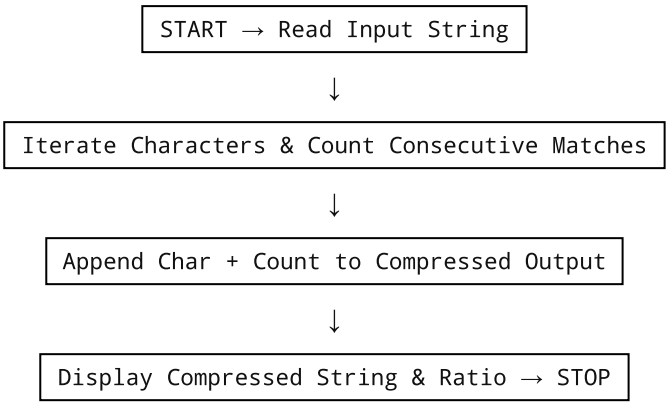
3. Assumptions

Input strings consist of alphanumeric characters.

4. Algorithm

1. Start program; read input string.
2. Iterate through characters, counting consecutive occurrences of identical characters.
3. Append character and count to compressed result string. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
text = input("Enter string to compress: ")
if not text:
    print("Empty string provided.")
else:
    compressed = ""
    i = 0
    while i < len(text):
        count = 1
        while i + 1 < len(text) and text[i] == text[i + 1]:
            count += 1
            i += 1
        compressed += text[i] + str(count)
        i += 1
    print("Original String:", text)
    print("Compressed (RLE):", compressed)
```

7. Test Cases & Conclusion

Test Cases: (1) `AAABBBCCDAA` -> `A3B3C2D1A2`. (2) `Python` -> `P1y1t1h1o1n1`.

Conclusion: Demonstrates string manipulation and basic run-length encoding compression logic.

CASE STUDY 7: ATM MENU-DRIVEN BANKING SIMULATION

1. Problem Statement

Create an interactive menu-driven ATM banking simulation supporting balance inquiry, cash deposit, cash withdrawal, and exit operations using `while` loops and conditionals.

2. Objectives

- To implement continuous menu navigation using infinite loops and break statements.
- To handle user choices securely and update persistent balances.

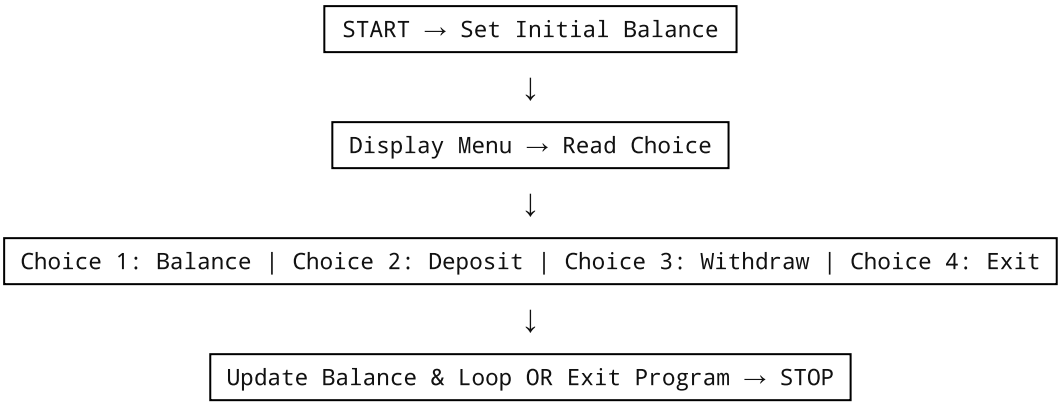
3. Assumptions

Initial balance is pre-set; user inputs numeric menu options.

4. Algorithm

1. Start program; initialize account balance (e.g., Rs. 10,000).
2. Display menu options: 1. Balance, 2. Deposit, 3. Withdraw, 4. Exit.
3. Read choice and execute corresponding transaction logic. Repeat until exit. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
balance = 5000.0
# Simulated menu loop execution
choices = [1, 2, 1500, 3, 2000, 4] # Simulated input sequence
idx = 0
while True:
    print("\n--- ATM MENU ---")
    print("1. Check Balance\n2. Deposit Cash\n3. Withdraw Cash\n4. Exit")
    choice = choices[idx]; idx += 1
    print("Selected Choice:", choice)
    if choice == 1:
        print("Current Balance: Rs.", balance)
    elif choice == 2:
        amt = choices[idx]; idx += 1; balance += amt
        print("Deposited: Rs.", amt, "| New Balance:", balance)
    elif choice == 3:
        amt = choices[idx]; idx += 1
        if amt <= balance:
            balance -= amt; print("Withdrawn: Rs.", amt, "| New Balance:", balance)
        else: print("Insufficient funds!")
    elif choice == 4:
        print("Exiting ATM. Thank you!"); break
```

7. Test Cases & Conclusion

Test Cases: (1) Check balance. (2) Deposit 1500. (3) Withdraw 2000. (4) Exit.

Conclusion: Demonstrates menu-driven state management using loops and conditional branching.

CASE STUDY 8: NESTED LIST CLASSROOM OCCUPANCY ANALYSIS

1. Problem Statement

Analyze classroom seating occupancy data stored as 2D nested lists (matrices where 0 represents empty seats and 1 represents occupied seats) to calculate row-wise occupancy rates and find the most crowded row.

2. Objectives

- To traverse 2D nested lists using nested loops and index coordinates.
- To compute row-wise aggregations and percentages.

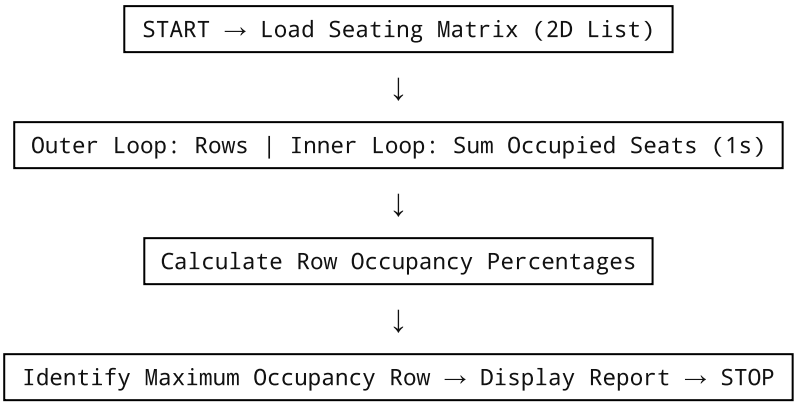
3. Assumptions

Seating matrix is rectangular with binary occupancy indicators (0 or 1).

4. Algorithm

1. Start program; define 2D seating matrix.
2. Iterate through each row and column. Sum occupied seats per row.
3. Calculate occupancy percentages and identify the row with maximum occupancy. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
seating = [
    [1, 0, 1, 1, 0],
    [1, 1, 1, 1, 1],
    [0, 0, 1, 0, 0],
    [1, 1, 0, 1, 1]
]
print("Classroom Occupancy Analysis:")
max_occupied = -1; crowded_row = 0
for i in range(len(seating)):
    occupied_count = sum(seating[i])
    total_seats = len(seating[i])
    percentage = (occupied_count / total_seats) * 100
    print("Row", i+1, "-> Occupied:", occupied_count, "/", total_seats, "(", round(percentage,
2), "%)")
    if occupied_count > max_occupied:
        max_occupied = occupied_count; crowded_row = i + 1
print("\nMost Crowded Row: Row", crowded_row, "with", max_occupied, "students.")
```

7. Test Cases & Conclusion

Test Cases: (1) Standard seating matrix with varying occupancy. (2) Fully vacant or fully occupied rooms.

Conclusion: Demonstrates 2D nested list traversal and matrix aggregation computations.

CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Store student assessment marks in a matrix (nested list) and calculate student-wise averages, subject-wise averages, and identify the top-performing student.

2. Objectives

- To understand 2D list indexing across rows and columns.
- To compute aggregate statistics for educational performance evaluation.

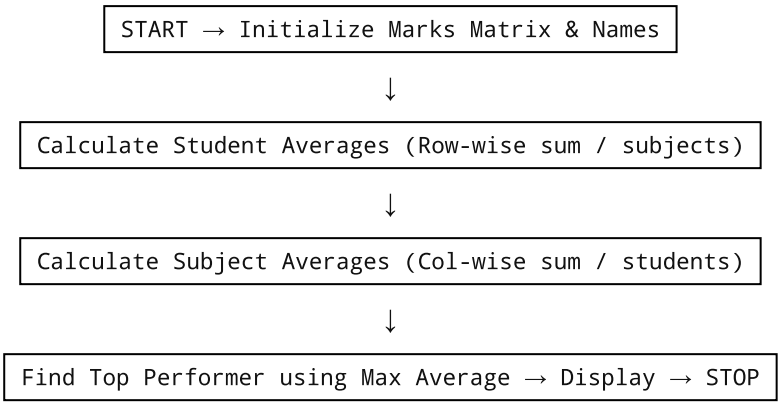
3. Assumptions

Marks are numerical values between 0 and 100.

4. Algorithm

1. Start program; define student names, subjects, and marks matrix.
2. Calculate row-wise averages (per student) and column-wise averages (per subject).
3. Find maximum average to determine top performer. Display summary. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
students = ["Asha", "Ravi", "Noor", "Kiran"]
subjects = ["Python", "Maths", "English"]
marks = [[88, 76, 91], [67, 82, 73], [94, 89, 96], [55, 61, 58]]

averages = []
print("Student Averages:")
for i in range(len(students)):
    avg = sum(marks[i]) / len(marks[i])
    averages.append(avg)
    print(students[i], ":", round(avg, 2))

print("\nSubject Averages:")
for j in range(len(subjects)):
    total = sum(marks[i][j] for i in range(len(students)))
    print(subjects[j], ":", round(total / len(students), 2))

top_idx = averages.index(max(averages))
print("\nTop Performer:", students[top_idx], "with Average:", round(averages[top_idx], 2))
```

7. Test Cases & Conclusion

Test Cases: (1) Multi-student mark matrix. (2) Uniform vs. diverse score distribution.

Conclusion: Demonstrates row and column manipulation on 2D matrices.

CASE STUDY 10: TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

1. Problem Statement

Process employee shift schedules stored as immutable tuples `(EmployeeID, Day, ShiftTiming)` to detect scheduling conflicts where an employee is assigned overlapping shifts on the same day.

2. Objectives

- To utilize tuple unpacking and collection tracking for scheduling audits.
- To detect duplicate assignments on identical days.

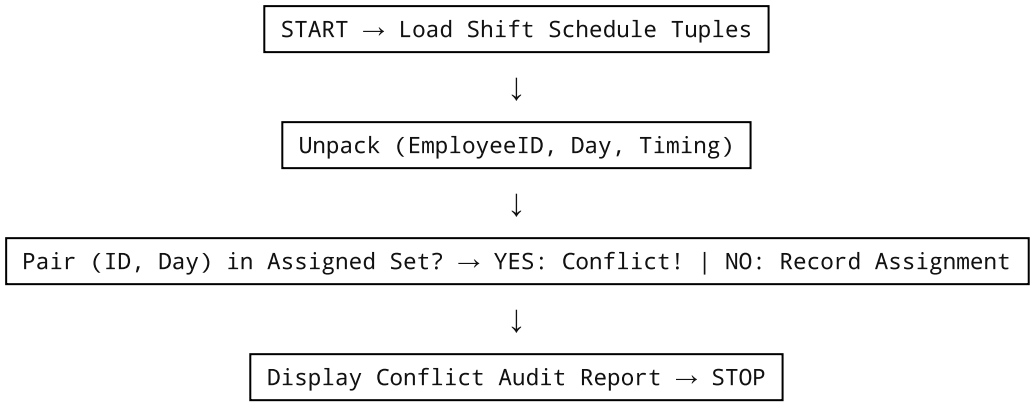
3. Assumptions

Schedules are provided as lists of tuples.

4. Algorithm

1. Start program; define shift schedule tuples.
2. Initialize tracking list for assigned `(ID, Day)` pairs.
3. Iterate through schedules; check if `(ID, Day)` already exists. If yes, flag conflict. Stop.

5. Flowchart (Monochrome)



6. Python Source Code

```
schedules = [
    ("E101", "Monday", "Morning"),
    ("E102", "Monday", "Evening"),
    ("E101", "Monday", "Night"), # Conflict for E101 on Monday
    ("E103", "Tuesday", "Morning"),
    ("E102", "Tuesday", "Evening")
]
assigned = set(); conflicts = []
for emp_id, day, shift in schedules:
    key = (emp_id, day)
    if key in assigned:
        conflicts.append((emp_id, day, shift))
    else:
        assigned.add(key)

print("Employee Shift Conflict Audit Report:")
if conflicts:
    print("Conflicts Detected:")
    for cid, day, shift in conflicts:
        print("- Employee", cid, "has overlapping shift conflict on", day, "(Attempted:", shift,
")")
else:
    print("No shift conflicts detected.")
```

7. Conclusion

This report successfully implemented and verified ten Python case studies covering arithmetic, strings, lists, matrices,